Midterm review

Chapter 1

Proposition: declarative statement that is either true or false

p	q	¬p	p∧q	p∨q	p⊕q	p→q	p↔q
T	T	F	T	T	F	T	T
T	F	F	F	T	T	F	F
F	T	T	F	T	T	T	F
F	F	T	F	F	F	T	T

converse: $q \to p$ (of $p \to q$)
contrapositive: $\neg q \to \neg p$ (of $p \to q$)
inverse: $\neg p \to \neg q$ (of $p \to q$)
truth tables 2^n rows ↳ no variables

if p, then q
if p, q
q unless ¬p
q if p
q whenever p
q follows from p
a necessary condition for p is q
a sufficient condition for q is p

p implies q
p only if q
q when p
p is sufficient for q
q is necessary for p

Precedence: ¬, ∧, ∨, →, ↔

* A list of propositions is consistent if it is possible to assign truth values to the proposition variables so that each proposition is true.

tautology: a proposition that is always true
contradiction: a proposition that is always false
contingency: a proposition that is sometimes true sometimes false

name	law	name	law
identity	$p \wedge T \equiv p$ $p \vee F \equiv p$	associative	$(p\wedge q)\wedge r \equiv p\wedge(q\wedge r)$
domination	$p \vee T \equiv T$ $p \wedge F \equiv F$	distributive	$p\vee(q\wedge r) \equiv (p\vee q)\wedge(p\vee r)$ $p\wedge(q\vee r) \equiv (p\wedge q)\vee(p\wedge r)$
idempotent	$p \vee p \equiv p$ $p \wedge p \equiv p$	demorgans	$\neg(p\wedge q) \equiv \neg p \vee \neg q$ $\neg(p\vee q) \equiv \neg p \wedge \neg q$
double negation	$\neg(\neg p) \equiv p$	absorption	$p\vee(p\wedge q) \equiv p$ $p\wedge(p\vee q) \equiv p$
commutative	$p\vee q \equiv q \vee p$ $p \wedge q \equiv q \wedge p$	negation	$p \vee \neg p \equiv T$ $p \wedge \neg p \equiv F$

DNF: disjunction of conjunctions
CNF: conjunction of disjunctions

* A compound proposition is satisfiable if there is an assignment of truth values that make it true. If not, then it's unsatisfiable.
↳ a compound proposition is unsatisfiable if & only if its negation is a tautology.

Notation
$$\bigvee_{j=1}^{n} p_j \equiv p_1 \vee p_2 \vee \ldots \vee p_n$$
$$\bigwedge_{j=1}^{n} p_j \equiv p_1 \wedge p_2 \wedge \ldots \wedge p_n$$

Predicate logic:
- Variables: x, y, z
- Predicate: P(x), M(x)
- Quantifiers

Propositional functions:
- generalization of propositions
- becomes a proposition when their variables are replaced by a value from the domain.

ex:
$P(x): x > 0$
$P(-3) \equiv F,\ P(0) \equiv F,\ P(3) \equiv T$

ex:
$R(x,y,z): x+y=z$
$R(2,-1,5) \equiv F$
$R(3,4,7) \equiv T$

compound expression:
ex:
$P(x): x>0$
$P(3) \vee P(-1) \equiv T$
$P(x) \to P(y)$

Quantifiers:
$\forall x\, P(x)$ (universal)
↳ P(x) is true for any x in the domain
$\exists x\, P(x)$ (existential)
↳ P(x) is true for at least one x in the domain
$\exists! x\, P(x)$ (uniqueness)
↳ is true for one & only one x in the domain

* the quantifiers ∀ & ∃ have higher precedence than all the logical operators
ex: $\forall x P(x) \vee Q(x)$ means $(\forall x P(x)) \vee Q(x)$

U: {1,2,3} $\forall x P(x) \equiv P(1) \wedge P(2) \wedge P(3)$
↳ domain $\exists x P(x) \equiv P(1) \vee P(2) \vee P(3)$

$\neg \forall x J(x) \equiv \exists x \neg J(x)$ (symbolically equivalent)

De morgan's
↳ $\neg \exists x P(x) \equiv \forall x \neg P(x)$
$\neg \forall x P(x) \equiv \exists x \neg P(x)$

Valid: true in any possible interpretation
Satisfiable: true in at least one possible interpretation
unsatisfiable: never true in any possible interpretation

Prolog
teaches(P,S) :- instructor(P,C), enrolled(S,C)
↳ Rule
instructor(ali, csi2101)
enrolled(sara, csi2101) } given facts
teaches(ali, sara) ↳ conclusion
?enrolled(Kevin, math2733) ↳ Query
yes → answer

nested quantifiers:
a quantifier inside the scope of another quantifier
ex:
$\forall x \exists y\, (x+y=0)$

$\forall x \forall y\, P(x,y)$
↳ for each x:
for each y:
check P(x,y)

$\forall x \exists y\, P(x,y)$
↳ for each x:
find one y s.t. $P(x,y) \equiv T$

* $\forall x \forall y P(x,y) \equiv \forall y \forall x P(x,y)$
but $\forall x \exists y Q(x,y) \not\equiv \exists y \forall x Q(x,y)$

$\forall x (P(x) \wedge Q(x)) \equiv \forall x P(x) \wedge \forall x Q(x)$
$\forall x (P(x) \to Q(x)) \equiv \forall x P(x) \to \forall x Q(x)$

Valid arguments: → if premises imply the conclusion, so $(P_1 \wedge P_2 \wedge \ldots P_n) \to q$ is a tautology
1. Propositional logic
Inference rules
2. Predicate logic
Inference rules for propositional logic plus additional inference rules to handle variables & quantifiers

argument: sequence of propositions. All but the final proposition are called premises. The last statement is the conclusion.

Rule of inference	tautology	name
$\frac{p \quad p\to q}{\therefore q}$	$(p \wedge (p\to q)) \to q$	modus ponens
$\frac{\neg q \quad p\to q}{\therefore \neg p}$	$(\neg q \wedge (p \to q)) \to \neg p$	modus tollens
$\frac{p\to q \quad q \to r}{\therefore p \to r}$	$((p\to q)\wedge(q\to r)) \to (p \to r)$	hypothetical syllogism
$\frac{p \vee q \quad \neg p}{\therefore q}$	$((p\vee q) \wedge \neg p) \to q$	disjunctive syllogism
$\frac{p}{\therefore p \vee q}$	$p \to (p \vee q)$	addition
$\frac{p \wedge q}{\therefore p}$	$(p \wedge q) \to p$	simplification
$\frac{p \quad q}{\therefore p \wedge q}$	$((p)\wedge(q)) \to (p \wedge q)$	conjunction
$\frac{p \vee q \quad \neg p \vee r}{\therefore q \vee r}$	$((p\vee q)\wedge(\neg p \vee r)) \to (q \vee r)$	resolution

valid argument: $S_1, S_2, \ldots, S_n \therefore C$

Universal instantiation (UI)
$\frac{\forall x P(x)}{\therefore P(c)}$ ex: D: all dogs Fido is a dog
"All dogs are cuddly"
"∴ Fido is cuddly."

Universal Generalization (UG)
$\frac{P(c) \text{ for an arbitrary } c}{\therefore \forall x P(x)}$ ex: show that for every integer n, n+n is even
c = 2k
↳ arbitrary integer

Existential instantiation (EI)
$\frac{\exists x P(x)}{\therefore P(c) \text{ for some element } c}$ ex: "∃ someone who got an A in this course"
"let's call her a & say that a got an A".

Existential generalization (EG)
$\frac{P(c) \text{ for some element } c}{\therefore \exists x P(x)}$ ex: "Michelle got an A in the class"
"Therefore, someone got an A in the class"

Universal modus ponens
$\forall x (P(x) \to Q(x))$
$P(a)$
$\therefore Q(a)$
a is a particular element in the domain

proof: valid argument that establishes the truth of a statement.
theorem: statement that can be shown using definitions, other theorems, axioms & rules of inference.
lemma: 'helping theorem' or a result which is needed to prove a theorem
corollary: result which follows directly from a theorem.
conjecture: statement that is being proposed to be true.

odd integer: $n = 2k+1$ $n, k \in \mathbb{Z}$
even integer: $n = 2k$

Direct proof: assume p is true & show that q must be true as well
$p \to q$

rational number: $r = p/q$ ↳ integers

Proof by contraposition: assume ¬q is true & show ¬p is true as well
$p \to q$

Proof by contradiction: to prove p assume ¬p → (p)
assume $p \wedge \neg q$ for $p \to q$

Pigeonhole principle:
if kn+1 objects are placed in n boxes, then at least one box contains k+1 or more objects.

Prove biconditional statement:
$p \leftrightarrow q$
we prove $p \to q$ & $q \to p$

proof by cases:
$P_1 \vee P_2 \vee \ldots \vee P_n$
$P_i \to R$
WLOG ↗ best way

existence proofs:
$\exists x P(x)$
(find a specific value for which it's true)

* $\exists x \neg P(x) \equiv \neg \forall x P(x)$

uniqueness proof:
existence: we show that an element x with the property exists
uniqueness: we show that if $y \neq x$, then y does not have that property

Universally quantified assertions:
$\forall x P(x)$ ↳ for this we assume x is an arbitrary member of the domain.

Chapter 2

Sets

Set: unordered set of element
elements/members: objects in a set
∈: element of
∉: not element of

$S = \{x \mid P(x)\}$
[ ], [ ), ( ], ( )

$N = \{0,1,2,3,\ldots\}$ $Z = \{\ldots,-3,-2,-1,0,1,2,3,\ldots\}$
$Z^+ = \{1,2,3,\ldots\}$ R = set of real numbers
R^+ = set of positive real numbers
C = set of complex numbers
Q = set of rational numbers

universal set: U * sets can be elements of sets
empty set: ∅, { }

Set equality: 2 sets are equal if & only if they have the same elements. $\forall x (x \in A \leftrightarrow x \in B)$ $A \subseteq B$ & $B \subseteq A$

Subset: the set A is a subset of B, if & only if every element of A is also an element of B.
$A \subseteq B$ ↳ subset
$A \not\subseteq B$ ↳ not subset
$A \subset B$ ↳ proper subset $A \neq B$

Set cardinality: denoted |A|, number of (distinct) elements of A.
finite: n distinct elements in S where n is a nonnegative integer
↳ otherwise infinite

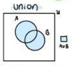
Power sets: denoted P(A), set of all subsets of a set A
* $|A| = n$ $|P(A)| = 2^n$

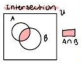
Tuples: ordered collection $(a_1, a_2, \ldots, a_n)$ ↳ first element, second element, n element
ordered pairs: (a, b)

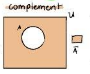
Cartesian product: $A \times B$ (a, b)
ex: A = {a, b} B = {1, 2, 3}
A×B = {(a,1),(a,2),(a,3),(b,1),(b,2),(b,3)}

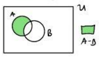
relation: a subset R of the cartesian product A×B.

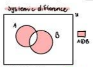
Truth set of quantifiers: $\{x \in D \mid P(x)\}$ ↳ domain ↳ predicate

Union


Intersection


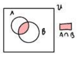
complement


Difference


systemic difference


A⊕B: A∪B − A∩B

cardinality of the union of two sets
inclusion–exclusion
$|A \cup B| = |A| + |B| - |A \cap B|$



Set identities

	Set identity	MAT1348 Name of law
1	$A \cup \varnothing = A$	Identity laws
2	$A \cap U = A$	
3	$A \cup U = U$	Domination laws
4	$A \cap \varnothing = \varnothing$	
5	$A \cup A = A$	Idempotent laws
6	$A \cap A = A$	
7	$\overline{(\overline{A})} = A$	Double complementation law
8	$A \cup B = B \cup A$	Commutative laws
9	$A \cap B = B \cap A$	
10	$(A\cup B)\cup C = A \cup (B \cup C)$	Associative laws
11	$(A\cap B)\cap C = A \cap (B \cap C)$	
12	$A\cap(B\cup C) = (A\cap B)\cup(A\cap C)$	Distributive laws
13	$A\cup(B\cap C) = (A\cup B)\cap(A\cup C)$	
14	$\overline{A\cup B} = \overline{A} \cap \overline{B}$	De morgan's laws
15	$\overline{A \cap B} = \overline{A} \cup \overline{B}$	
16	$A \cup (A \cap B) = A$	Absorption laws
17	$A \cap (A \cup B) = A$	
18	$A \cup \overline{A} = U$	Complement laws
19	$A \cap \overline{A} = \varnothing$	
20	$A - B = A \cap \overline{B}$	Difference law
21	$A \oplus B = (A-B)\cup(B-A)$	Symmetric difference laws
22	$A \oplus B = (A\cup B) - (A \cap B)$	

membership table rules
$1 \cap 1 \equiv 1$ $1 \cap 0 \equiv 0$ $0 \cap 0 \equiv 0$
$1 \cup 1 \equiv 1$ $1 \cup 0 \equiv 1$ $0 \cup 0 \equiv 0$

let $A_1, A_2, \ldots, A_n$ be an indexed collection of sets
union
$\bigcup_{i=1}^{n} A_i = A_1 \cup A_2 \cup \ldots \cup A_n$
intersection
$\bigcap_{i=1}^{n} A_i = A_1 \cap A_2 \cap \ldots \cap A_n$

Functions

Def: $f: A \to B$, assignment of each element of A to exactly one element of B.
also subset of A×B.
A: domain of f B: codomain of f
if $f(a) = b$ ↳ image of a under f ↳ preimage of b
range of f: set of all images of points in A under f. ↳ denoted by f(A)
equal functions: same domain & codomain, map each other in the same way.
injective function: if & only if ∀ for all a & b $f(a) = f(b)$ implies $a = b$
Surjective function: if & only if $\forall b \in B\ \exists a A$ s.t. $f(a) = b$ ↳ also called onto
bijection: injective & surjective ↳ also called a one-to-one correspondence, invertible
inverse function: f needs to be a bijection, inverse of $f = f^{-1}$
f^{-1} is $B \times A$, $f^{-1}(y) = x$ if $f(x) = y$
Composition:
$f: B \to C$ $g: A \to B$
$f \circ g = f(g(x))$ $A \to C$
floor function: $f(x) = \lfloor x \rfloor$ largest $\mathbb{Z}$ less or equal to x.
Ceiling function: $f(x) = \lceil x \rceil$ smallest $\mathbb{Z}$ greater than or equal to x.

* partial functions

Sequences & summations

Sequences: ordered lists of elements
↳ function from a subset of the integers to a set A

geometric progression: $a, ar^2, \ldots, ar^n, \ldots$
arithmetic progression: $a, a+d, a+2d, \ldots, a+nd, \ldots$
String: finite sequence of characters from a finite set (alphabet)

Recurrence relation: for the sequence $\{a_n\}$ is an equation that expresses a_n in terms of one or more of the previous terms of the sequence. * initial conditions

ex:
Fibonacci sequence
$f_0 = 0, f_1 = 1$ ↗ initial condition
$f_n = f_{n-1} + f_{n-2}$ ↗ recurrence relation

Summation
$\sum_{j=m}^{n} a_j$ $\sum_{j=m}^{n} a_j$ $\sum_{m \le j \le n} a_j$
represents $a_m + a_{m+1} + \cdots + a_n$
more generally for a set S:
$\sum_{j \in S} a_j$
ex:
$r^0 + r^1 + r^2 + r^3 + \cdots + r^n = \sum_0^n r^j$
$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots = \sum_1^\infty \frac{1}{i}$
if $S = \{2,5,7,10\}$ then $\sum_{j \in S} a_j = a_2 + a_5 + a_7 + a_{10}$

Product
$\prod_{j=m}^{n} a_j$ $\prod_{j=m}^{n} a_j$ $\prod_{m \le j \le n} a_j$
represents: $a_m \times a_{m+1} \times \cdots \times a_n$

geometric series
$$\sum_{j=0}^{n} ar^j = \begin{cases} \frac{ar^{n+1} - a}{r-1} & r \neq 1 \\ (n+1)a & r = 1 \end{cases}$$

sum	closed form	sum	closed form
$\sum_{k=0}^{n} ar^k\ (r \neq 0)$	$\frac{ar^{n+1}-a}{r-1}, r \neq 1$	$\sum_{k=1}^{n} k^3$	$\frac{n^2(n+1)^2}{4}$
$\sum_{k=1}^{n} k$	$\frac{n(n+1)}{2}$	$\sum_{k=0}^{\infty} x^k, \lvert x\rvert<1$	$\frac{1}{1-x}$
$\sum_{k=1}^{n} k^2$	$\frac{n(n+1)(2n+1)}{6}$	$\sum_{k=1}^{\infty} kx^{k-1}, \lvert x\rvert<1$	$\frac{1}{(1-x)^2}$

Set cardinality

$|A| = |B| \leftrightarrow$ bijection from A to B
injection from A to B $\to |A| \le |B|$
countable: finite or has same cardinality of (Z^+) else uncountable
* An infinite set is countable if & only if you can list its elements in a sequence
* Positive rational numbers are countable
* real numbers are uncountable
Computable: ↗ for functions if there is a computer program in some programming language that finds the values of this function.

Reflexive
a relation R on set A
$\forall x, (x \in A) \to ((x,x) \in R)$
or
$\forall x, (x \in A) \to (x R x)$

Symmetric
$\forall x,y \in A, ((x,y) \in R) \to ((y,x) \in R)$
or
$\forall x,y \in, (x R y) \to (y R x)$

Antisymmetric
$\forall x,y \in A, ((x,y) \in R \wedge (y,x) \in R) \to (x=y)$
or
$\forall x,y \in A, (x R y \wedge y R x) \to (x = y)$
or
$\forall x,y \in A, (x \neq y) \to ((x,y) \notin R \text{ or } (y,x) \notin R)$

Transitive
$\forall x,y,z \in A, ((x,y) \in R \wedge (y,z) \in R) \to ((x,z) \in R)$
or
$\forall x,y,z \in A, (x R y \wedge y R z) \to (x R z)$

Matrices

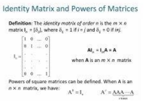
def: rectangular array of numbers
a matrix with m rows & n columns is an m×n matrix
↳ m=n called a square

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

addition
* matrices need to have the same size
$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 2 & -3 \\ 3 & 4 & 0 \end{bmatrix} + \begin{bmatrix} 3 & 4 & -1 \\ 1 & -3 & 0 \\ -1 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 4 & -2 \\ 3 & -1 & -3 \\ 2 & 5 & 2 \end{bmatrix}$$

multiplication
$$\underset{4\times3}{\begin{bmatrix} 1 & 0 & 4 \\ 2 & 1 & 1 \\ 3 & 1 & 0 \\ 0 & 2 & 2 \end{bmatrix}} \underset{3\times2}{\begin{bmatrix} 2 & 4 \\ 1 & 1 \\ 3 & 0 \end{bmatrix}} = \underset{4\times2}{\begin{bmatrix} 14 & 4 \\ 8 & 9 \\ 7 & 13 \\ 8 & 2 \end{bmatrix}}$$

* $AB \neq BA$

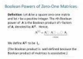

Identity Matrix and Powers of Matrices
Definition: The identity matrix of order n is the m × n matrix $I_n = [\delta_{ij}]$, where $\delta_{ij} = 1$ if $i = j$ and $\delta_{ij} = 0$ if $i \neq j$.
$AI_n = I_mA = A$ when A is an m × n matrix
Powers of square matrices can be defined. When A is an n × n matrix, we have: $A^0 = I_n$ $A^r = \underbrace{AAA\cdots A}_{r \text{ times}}$

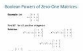
Transpose
$A \to A^T$
rows → columns
Columns → rows
$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Square matrix is symmetric if $A = A^t$

Zero-one matrices: all entries are 0 or 1
↳ join of A & B ↗ m×n
$a_{ij} \vee b_{ij}$ $A \vee B$
↳ meet of A & B
$a_{ij} \wedge b_{ij}$ $A \wedge B$
ex:
$A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ $A \vee B = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}$
$B = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}$ $A \wedge B = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$

Boolean product (zero-one)
$A = [a_{ij}]$ m×k $B = [b_{ij}]$ k×n
$A \odot B$
$c_{ij} = (a_{i1} \wedge b_{1j}) \vee (a_{i2} \wedge b_{2j}) \vee \ldots$
↳ same $\vee (a_{ik} \wedge b_{kj})$
ex:
$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$ $B = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$
$A \odot B = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}$


Boolean Powers of Zero-One Matrices
Definition: Let A be a square zero-one matrix and let r be a positive integer. The rth Boolean power of A is the Boolean product of r factors of A, denoted by $A^{[r]}$. Hence, $A^{[r]} = \underbrace{A \odot A \odot \ldots \odot A}_{r \text{ times}}$
We define $A^{[0]}$ to be I_n.
(The Boolean product is well defined because the Boolean product of matrices is associative.)


Boolean Powers of Zero-One Matrices
Example: Let
Find A^n for all positive integers n.
Solution:

Chapter 3

maximum finding: compare one element at a time & update max only when needed
Search algorithms: show the inspected element or the remaining binary-search interval.
Sorting algorithms: show comparisons, swaps, inserts, splits, merges, and pivots step by step.
greedy algorithms: show the local choice at each step & the remaining problem.
algorithm: finite sequence of precise steps that solves a problem
Pseudocode: written for humans, precise enough to analyze, but not tied to one programming language.

Input: algorithm has input values from a specified set
output: from the input, the algorithm produces the output values (solution)
correctness: should produce correct output
finiteness: should produce output after a finite # of steps for any input.
effectiveness: must be possible to perform each step correctly & in a finite amount of time.
generality: should work for all problems of the desired form.

max finding rules
1. Set max to first element
2. Scan the list from left to right, one element at a time
3. Update max only when the current element is larger.
4. After last comparison, max is the largest value.

Goal: find largest value in a list

linear search rules
1. Checks elements in order from left to right
2. Stop immediately when the target is found
3. If the end is reached, the target is absent.
4. worst case: every element must be examined

Goal: determine whether a target value appears in a list.

Binary search rules
1. list must already be sorted
2. Compare the target with the middle element
3. Discard the left half or right half after each comparison
4. Stop when the target is found or the interval becomes empty

Goal: find a target in a sorted list
$m = (i+j)/2$

ex:
[2, 5, 8, 11, 14, 17, 20, 23] target: 17
① middle: 11 17 > 11
② [14, 17, 20, 23] middle: 17 target found

ex:
[1, 4, 7, 10, 13, 16, 19, 22] target: 15
① middle: 10 15 > 10
② [13, 16, 19, 22] middle: 16 15 < 16
③ [13] middle: 13 15 > 13 (empty interval)

Bubble sort
concept
- compare adjacent pairs from left to right
- Swap the pair when the left value is larger than the right value
- After one full pass, the largest remaining elements "bubbles" to its final position at the right.
- repeat passes until the array is sorted

Rules
1. Compare adjacent elements from left to right.
2. Swap the pair if the left value is greater than the right value.
3. After each full pass, the largest remaining value moves to the right.
4. repeat passes until no more swaps are needed

ex:
[9, 4, 7, 1]
① [4, 7, 1, 9] ② [4, 1, 7, 9] ③ [1, 4, 7, 9]

ex:
[5, 2, 8, 3, 6]
① [2, 5, 3, 6, 8] ② [2, 3, 5, 6, 8]

ex:
[2, 8, 5, 3, 9, 4, 1]
① [2, 5, 3, 8, 4, 1, 9] ② [2, 3, 5, 4, 1, 8, 9]
③ [2, 3, 4, 1, 5, 8, 9] ④ [2, 3, 1, 4, 5, 8, 9]
⑤ [2, 1, 3, 4, 5, 8, 9] ⑥ [1, 2, 3, 4, 5, 8, 9]

Θ(n²)

Insertion sort
concept
- Treat the first element as a sorted prefix
- take the next element & insert it into the correct position within the prefix.
- After processing the first j elements, those j elements are sorted relative to eachother
- Insertion sort is often efficient on nearly sorted data

rules
1. Treat the left part as a sorted prefix.
2. Take the next unprocessed value.
3. Shift larger values to the right & insert the new value in the correct position.
4. After each step, the sorted prefix grows by one element.

ex:
[4, 11, 7, 5, 13, 6] [4, 5, 7, 11, 13, 6]
[4, 7, 11, 5, 13, 6] [4, 5, 6, 7, 11, 13]
[4, 5, 7, 11, 13, 6]

ex:
[6, 1, 4, 2] [1, 4, 6, 2]
[1, 6, 4, 2] [1, 2, 4, 6]

ex:
[10, 3, 7, 5, 2] [3, 5, 7, 10, 2]
[3, 10, 7, 5, 2] [2, 3, 5, 7, 10]
[3, 7, 10, 5, 2]

Θ(n²)

Selection sort
Concept
- Search the unsorted part of the array for the smallest remaining element
- Swap that smallest element into the first position of the unsorted part.
- After pass 1, the first element is final; after pass 2, the first two elements are final.
- The algorithm performs many comparisons even if the array is nearly sorted.

rules
1. Find the smallest value in the unsorted part
2. Swap it with the first position of the unsorted part
3. After each pass, one more position on the left is fixed
4. If the minimum is already in place, no real change is needed.

ex:
[6, 3, 8, 2, 7, 5, 1] [1, 2, 3, 5, 7, 8, 6]
[1, 3, 8, 2, 7, 5, 6] [1, 2, 3, 5, 6, 8, 7]
[1, 2, 8, 3, 7, 5, 6] [1, 2, 3, 5, 6, 7, 8]
[1, 2, 3, 8, 7, 5, 6]

ex:
[8, 6, 3, 9, 1] [1, 3, 6, 9, 8]
[1, 6, 3, 9, 8] [1, 3, 6, 8, 9]
[1, 3, 6, 9, 8]

ex:
[4, 10, 2, 7, 5] [2, 4, 5, 7, 10]
[2, 10, 4, 7, 5] [2, 4, 5, 7, 10]
[2, 4, 10, 7, 5]

Θ(n²)

Merge sort
Concept
- Divide the list into two halves.
- recursively sort each half.
- merge two sorted lists by repeatedly taking the smaller front element.
- merge sort is a divide-and-conquer algorithm with Θ(n log n) running time.

rules
1. Split the list into two halves until small lists are obtained.
2. Sort each half recursively.
3. merge two sorted lists by repeatedly taking the smaller front value.
4. the merge phase rebuilds the final sorted list

ex:
[6, 2, 9, 4, 8, 1, 7, 3] → [6, 2, 9, 4] [8, 1, 7, 3]
→ [6, 2] [9, 4] [8, 1] [7, 3]
→ [6] [2] [9] [4] [8] [1] [7] [3]
→ [2, 6] [4, 9] [1, 8] [3, 7]
→ [2, 4, 6, 9] [1, 3, 7, 8]
→ [1, 2, 3, 4, 6, 7, 8, 9]

ex:
[9, 2, 6, 4, 1, 7] → [9, 2, 6] [4, 1, 7]
→ [9] [2, 6] [4] [1, 7] → [2, 6, 9] [1, 4, 7]
→ [1, 2, 4, 6, 7, 9]

ex:
[12, 5, 3, 10, 8, 1, 6, 4] → [12, 5, 3, 10] [8, 1, 6, 4]
→ [3, 5, 10, 12] [1, 4, 6, 8] → [1, 3, 4, 5, 6, 8, 10, 12]

Split phase
[9, 2, 6, 4, 1, 7]
[9, 2, 6] [4, 1, 7]
[9] [2, 6] [4] [1, 7]
[9] [2] [6] [4] [1] [7]

Merge phase
merge [2] & [6] [2, 6]
merge [1] & [7] [1, 7]
merge [9] & [2, 6] [2, 6, 9]
merge [4] & [1, 7] [1, 4, 7]
Final merge [2, 6, 9] & [1, 4, 7] [1, 2, 4, 6, 7, 9]

final merge details
merge [3, 5, 10, 12] & [1, 4, 6, 8]
1. compare 3 & 1, take 1, [1]
2. compare 3 & 4, take 3, [1, 3]
3. compare 5 & 4, take 4, [1, 3, 4]
4. compare 5 & 6, take 5, [1, 3, 4, 5]
5. compare 10 & 6, take 6, [1, 3, 4, 5, 6]
6. compare 10 & 8, take 8, [1, 3, 4, 5, 6, 8]
7. left side remains, append 10, 12 [1, 3, 4, 5, 6, 8, 10, 12]

Θ(n log n)

Quick sort
concept
- Choose a pivot value.
- partition the array so that smaller values go before the pivot & larger values go after it.
- recursively quick-sort the left & right parts
- A partition result is valid if it respects the pivot rule; the values inside each side do not need to be sorted yet.

rules
1. Choose a pivot.
2. After partioning, the pivot is in the correct final position.
3. Items to the left of the pivot are smaller.
4. Items to the right of the pivot are larger.
5. recursively apply quick sort to the left & right parts.

ex:
[7, 2, 9, 1, 6, 4] 7
[2, 1, 6, 4] [7] [9]
[1] [2, 6, 4] [7] [9]
[1] [2] [6, 4] [7] [9]
[1] [2] [4] [6] [7] [9]
[1, 2, 4, 6, 7, 9]

ex:
[11, 5, 14, 3, 9, 2] 9
[5, 3, 2] [9] [11, 14] [2, 3, 5, 9, 11, 14]
[3, 2] [5] [9] [11, 14]
[2] [3] [5] [9] [11, 14]
[2] [3] [5] [9] [11] [14]

Average Θ(n log n)

Greedy algorithms
concept
- chooses what looks best right now
- local optimal choice
↳ sometimes lead to a globally optimal solution (or not)
- to trust this type of algorithm we must prove it always gives an optimal solution for that problem.

Greedy coin change rules
1. At each step, choose the largest coin that does not exceed the remaining amount.
2. Subtract that coin value from the remaining amount.
3. repeat until the remaining amount is zero.
4. Greedy may fail for some coin systems, so a proof or counterexample is needed.

Greedy scheduling: activity selection
concept
- problem: choose as many compatible tasks/activities as possible
- two activities are compatible if their time intervals do not overlap
- greedy strategy: repeatedly choose the compatible activity that ends earliest
- why this works: ending early leaves the most room for future activities

rule
1. Sort or consider activities by earliest finishing time
2. choose the activity that ends earliest among compatible choices
3. remove activities that overlap the selected one
4. repeat to leave as much room as possible for future activities

Halting problem
Concept:
- asks whether there is one general algorithm that can decide if any program will eventually stop
- * no such algorithm exists
* which means there is no universal solution for all programs & all inputs.

asymptotic notation
Big-O: asymptotic upper bound (worst case) O(n)
Big-omega: asymptotic lower bound (best case) Ω(n)
Big-Theta: tight asymptotic bound (in between) (exact) Θ(n)

* n log n is O(n²)

nested loops rules
① count how many times inner statement executes
② for equal bounds n by n, multiply n×n
③ for triangular loops, add 1+2+…+n
④ keep the dominant term for asymptotic complexity.

* average-case complexity of linear search is Θ(n)
* worst-case complexity of binary search O(log n)
* worst-case complexity of bubble sort n²
* worst-case complexity insertion sort n²
* Complexity of finding A ⊙ B is n³
& matrix multiplication

algorithmic paradigm:
general approach based on a particular concept for constructing algorithms to solve a variety of problems.

brute-force algorithm:
most straightforward manner, with thinking about efficiency.
ex: sequential search, bubble sort, insertion sort

Complexity	terminology
Θ(1)	constant
Θ(log n)	logarithmic
Θ(n)	linear
Θ(n log n)	linearithmic
Θ(n^b)	Polynomial
Θ(b^n), where b > 1	exponential
Θ(n!)	factorial

matrix multiplication
① n×n result matrix has n² entries
② each entry is computed using n multiply-add terms.
③ Total work is n²×n = n³
④ therefore, the standard algorithm has O(n³) time complexity.

Chapter 4

Division

Def: if a & b are integers with $a \neq 0$, then a divides b if there exists an integer c s.t. $b = ac$.

if a divides b then $a|b$ else $a \nmid b$

theorem $a \neq 0$

1) if $a|b$ & $a|c$, then $a|(b+c)$
2) if $a|b$, then $a|bc$
3) if $a|b$ & $b|c$, then $a|c$

Division algorithm

$a = dq + r \rightarrow$ remainder

a → dividend, d → divisor, q → quotient

$q = a \text{ div } d$

$r = a \bmod d$

Congruence relation

$a, b \in \mathbb{Z}$ & $m \in \mathbb{Z}^+$

a is congruent to b modulo m if $m|(a-b)$

so $a \equiv b \pmod{m}$ → modulus; → congruence (a & b)

* 2 integers are congruent mod m if & only if they have the same remainder when divided by m

& if $a = b + km$ → is an integer that exists; $a \equiv b \pmod m$

$a \equiv b \pmod m \longleftrightarrow a \bmod m = b \bmod m$

if $a \equiv b \pmod m$ & $c \equiv d \pmod m$, then

$a + c \equiv b + d \pmod m$ &

$ac \equiv bd \pmod m$

if $a \equiv b \pmod m$, then $ca \equiv cb \pmod m$

if $a \equiv b \pmod m$, then $c + a \equiv c + b \pmod m$

$(a+b) \pmod m = ((a \bmod m) + (b \bmod m)) \bmod m$

$ab \bmod m = ((a \bmod m)(b \bmod m)) \bmod m$

Arithmetic modulo m

$a +_m b = (a+b) \bmod m$ * addition modulo m

$a \cdot_m b = (a \cdot b) \bmod m$ * multiplication modulo m

- $(a +_m b) +_m c = a +_m (b +_m c)$
- $(a \cdot_m b) \cdot_m c = a \cdot_m (b \cdot_m c)$
- $a +_m b = b +_m a$ & $a \cdot_m b = b \cdot_m a$
- $a +_m 0 = a$ & $a \cdot_m 1 = a$
- $a +_m (m-a) = 0$ & $0 +_m 0 = 0$
- $a \cdot_m (b +_m c) = (a \cdot_m b) +_m (a \cdot_m c)$ &
- $(a +_m b) \cdot_m c = (a \cdot_m c) +_m (b \cdot_m c)$

Base b representations

$n = a_k b^k + a_{k-1} b^{k-1} + \dots + a_1 b + a_0$

Binary expansion: binary → decimal

octal expansion: octal → decimal

Hexadecimal expansion: Hexadecimal → decimal

Base conversion: base b, integer n; divide n by b collect remainder keep dividing quotient until its 0.

Binary modular exponentiation

$n = (a_{k-1}, \dots, a_1, a_0)_2$

Compute b^n

$b^n = b^{a_{k-1} \cdot 2^{k-1} + \dots + a_1 \cdot 2 + a_0}$

$= b^{a_{k-1} \cdot 2^{k-1}} \cdots b^{a_1 \cdot 2} \cdot b^{a_0}$

11/2	5	1
5/2	2	1
2/2	1	0
1/2	0	1

$3^{11} = 3^8 \cdot 3^2 \cdot 3^1 = ((3^2)^2)^2 \cdot 3^2 \cdot 3^1$

$11 = (1011)_2$

$= (9^2)^2 \cdot 9 \cdot 3$

$= (81)^2 \cdot 9 \cdot 3$

$= 6561 \cdot 9 \cdot 3$

$= 177147$

Prime

Def: positive $\mathbb{Z} > 1$ is called prime if the only positive factors of p are 1 & p else, its composite

* every positive integer greater than 1 can be written uniquely as a prime or as the product of two or more primes where the prime factors are written in order of nondecreasing size

Sieve of erastosthenes

1. $\forall x \in \mathbb{Z}, x \neq 2$ but $2|x$ out
2. $\forall x \in \mathbb{Z}, x \neq 3$ but $3|x$ out
3. $\forall x \in \mathbb{Z}, x \neq 5$ but $5|x$ out
4. $\forall x \in \mathbb{Z}, x \neq 7$ but $7|x$ out

if an integer n is a composite integer, then it has prime divisor less than or equal to $\sqrt{n}$

* if $n = ab$, then $a \leq \sqrt{n}$ or $b \leq \sqrt{n}$

trivial division: try every integer $i \leq n$ & see if $i|n$

Prime numbers like: $2^p - 1$ are called Mersenne primes

Prime number theorem: the ratio of the number of primes not exceeding x & $x/\ln x$ approaches 1 as x grows without bound

GCD

largest integer d s.t. $d|a$ & $d|b$ which is the GCD of a & b

* 2 $\mathbb{Z}$ are relatively prime if their GCD is 1.

* the $\mathbb{Z}$ $a_1, a_2, \dots, a_n$ are pairwise relatively prime if $\gcd(a_i, a_j) = 1$ whenever $1 \leq i < j \leq n$

ex: 10, 17, 21 are pairwise prime

$\gcd(10,17)=1$ $\gcd(10,21)$ $\gcd(17,21)$

finding the gcd using prime factorizations

$a = p_1^{a_1} p_2^{a_2} \cdots p_n^{a_n}$ $b = p_1^{b_1} p_2^{b_2} \cdots p_n^{b_n}$

$\gcd(a,b) = p_1^{\min(a_1,b_1)} p_2^{\min(a_2,b_2)} \cdots p_n^{\min(a_n,b_n)}$

$\gcd(120,500) = 2^{\min(3,2)} \cdot 3^{\min(1,0)} \cdot 5^{\min(1,3)}$

$120 = 2^3 \cdot 3 \cdot 5$ $= 2^2 \cdot 3^0 \cdot 5^1 = 20$

$500 = 2^2 \cdot 5^3$

least common multiple

$\operatorname{lcm}(a,b) = p_1^{\max(a_1,b_1)} p_2^{\max(a_2,b_2)} \cdots p_n^{\max(a_n,b_n)}$

$\operatorname{lcm}(2^3 \cdot 3^5 \cdot 7^2, 2^4 \cdot 3^3) = 2^{\max(3,4)} 3^{\max(5,3)} 7^{\max(2,0)}$

$= 2^4 \cdot 3^5 \cdot 7^2$

Euclidian algorithm

base on idea that $\gcd(a,b) = \gcd(a,c)$ when $a > b$ & c is the remainder when a is divided by b.

$\gcd(91, 287)$:

$287 = 91 \cdot 3 + 14$

$91 = 14 \cdot 6 + 7$

$14 = 7 \cdot 2 + 0$

$\gcd(287,91) = \gcd(91,14) = \gcd(14,7) = 7$

* let $a = bq + r$ then $\gcd(a,b) = \gcd(b,r)$

Bézout's theorem

$\gcd(a,b) = sa + tb$ } Bézout's identity; $\mathbb{Z}^+$; Bézout coefficients

ex: $\gcd(6,14) = (-2) \cdot 6 + 1 \cdot 14$

ex: $\gcd(252,198) = 18$

$18 = 4 \cdot 252 - 5 \cdot 198$

* if $\gcd(a,b) = 1$ & $a|bc$, then $a|c$

* if p is prime & $p | a_1 a_2 \cdots a_n$, then $p | a_i$

Theorem: if $ac \equiv bc \pmod m$ & $\gcd(c,m) = 1$, then $a \equiv b \pmod m$

linear congruences

congruence of the form: $ax \equiv b \pmod m$ (a, b → $\mathbb{Z}$; m → $\mathbb{Z}^+$; x → variable called linear congruence)

→ the solutions to it are all integers x that satisfy the congruence

* an integer $\bar{a}$ s.t. $\bar{a}a \equiv 1 \pmod m$ is said to be an inverse of a modulo m.

ex: 5 is an inverse of 3 modulo 7

$5 \cdot 3 \equiv 1 \pmod 7$

* if $\gcd(a,m) = 1$ & $m > 1$, then an inverse of a modulo m exists. & this inverse is unique

finding inverses

ex: find an inverse of 3 modulo 7

$\gcd(3,7) = 1$

euclidian algorithm: $7 = 2 \cdot 3 + 1$

$-2 \cdot 3 + 1 \cdot 7 = 1$ ← Bézout coefficients of 3 & 7

hence -2 is an inverse of 3 modulo 7. every integer congruent to -2 modulo 7 is an inverse of 3 modulo 7, i.e., 5, -9, 12, etc.

$ax \equiv b \pmod m$

→ can be solved by multiplying $\bar{a}$ by both sides

$-2 \cdot 3x \equiv -2 \cdot 4 \pmod 7$

$-6 \equiv 1 \pmod 7$ & $-8 \equiv 6 \pmod 7$

$x \equiv -8 \equiv 6 \pmod 7$

$3x \equiv 3 \cdot 6 = 18 \equiv 4 \pmod 7$

*** mersenne prime**

$2^p - 1$

Prime is p & $2^p - 1$ is Prime

Fermat's little theorem

if p is prime & $p \nmid a$ then,

$a^{p-1} \equiv 1 \pmod p$

$a^p \equiv a \pmod p$

ex: $7^{222} \bmod 11$

$(7^{10})^{22} 7^2$

$\equiv (1)^{22} \cdot 49$

$\equiv 5 \pmod{11}$

$7^{222} \bmod 11 = 5$

Chinese remainder theorem

let $m_1, m_2, \dots, m_n$ be pairwise relatively prime positive integers greater than one & $a_1, a_2, \dots, a_n$ arbitrary integers.

$x \equiv a_1 \pmod{m_1}$

$x \equiv a_2 \pmod{m_2}$

$\vdots$

$x \equiv a_n \pmod{m_n}$

has unique solution modulo $m = m_1 m_2 \cdots m_n$; solution x with $0 \leq x < m$

$M_k = m/m_k$ ($m \to m_1 m_2 \cdots m_n$; $k \to 1, 2, \dots, n$)

since $\gcd(m_k, M_k) = 1$,

$M_k y_k \equiv 1 \pmod{m_k}$

$x = a_1 M_1 y_1 + a_2 M_2 y_2 + \dots + a_n M_n y_n$

$x = 2 \cdot 35 \cdot 2 + 3 \cdot 21 \cdot 1 + 2 \cdot 15 \cdot 1 = 233 \equiv 23 \pmod{105}$

ex:

$x \equiv 2 \pmod 3$ (a_1, m_1)

$x \equiv 3 \pmod 5$ (m_2)

$x \equiv 2 \pmod 7$ (m_3, a_3)

let $m = 3 \cdot 5 \cdot 7 = 105$

$M_1 = m/3 = 35$

$M_2 = m/5 = 21$

$M_3 = m/7 = 15$

2 is an inverse of M_1

1 is an inverse of M_2

1 is an inverse of M_3

Back substitution

$x \equiv 1 \pmod 5 \rightsquigarrow 5t + 1 = x$

$x \equiv 2 \pmod 6 \rightsquigarrow 6t + 2 = x$

$x \equiv 3 \pmod 7 \rightsquigarrow 7t + 3 = x$

$5t + 1 \equiv 2 \pmod 6$

$t \equiv 5 \pmod 6$

$t = 6u + 5$

$x = 5 \cdot (6u + 5) + 1$

$= 30u + 26$

$30u + 26 \equiv 3 \pmod 7$

$u \equiv 6 \pmod 7$

$u = 7v + 6$

$x = 30u + 26$

$x = 30(7v + 6) + 26 = 210v + 206$

$x \equiv 206 \pmod{210}$

Pseudoprimes

$n > 2$

$2^{n-1} \equiv 1 \pmod n$ → if n does not satisfy it is composite

→ may not be prime; so called pseudo prime to the base 2

Primitive roots

Primitive root modulo: prime p is an integer r in $\mathbb{Z}_p$ s.t every nonzero element of $\mathbb{Z}_p$ is a power of r

ex: $\mathbb{Z}_{11}$ is a power of 2, 2 is a primitive root of 11.

Powers of 2 modulo 11: $2^1=2, 2^2=4, 2^3=8, 2^4=5, 2^5=10, 2^6=9, 2^7=7, 2^8=3, 2^{10}=2$

Discrete logarithms

p = prime r = primitive root modulo p

$1 < a < p-1$ $a \in \mathbb{Z}_p$

$\exists e$ s.t $r^e = a$ in $\mathbb{Z}_p$

$r^e \bmod p = a$

ex: $\log_2 5 = 4$ since 5 mod 11 to the base 2 is 4

Applications of congruences

Diffie-Hellman key exchange

$(a^{k_2})^{k_1} \bmod p = (a^{k_1})^{k_2} \bmod p$ (Alice computes; Bob computes)

Alice RSA public key: (n, e)

Alice private key = d

$E_{(n,e)}(x) = x^e \bmod n$ → encrypt (x, y are messages)

$D_{(n,e)}(y) = y^d \bmod n$ → decrypt

ex: Alice:

Key (2537, 13) $2537 = 43 \cdot 59$

$p = 43$ & $q = 59$

$d = 937$

MEET AT NOON

1204 0419 0019 1314 1413

$D_{(2537,13)}(x) = x^{937} \bmod 2537$

Sends 0817 0555 1310 2173 1026

Cryptography

ceasar cipher

$f(p) = (p+3) \bmod 26$ } encryption

A = 0 B = 1, etc Z = 25

$f^{-1}(p) = (p-3) \bmod 26$ } decryption

Shift cipher

$f(p) = (p+k) \bmod 26$ } encryption

A = 0 B = 1, etc Z = 25

$f^{-1}(p) = (p-k) \bmod 26$ } decryption

affine cipher

$f(p) = (ap+b) \bmod 26$ } encryption

ex: $f(p) = (7p+3) \bmod 26$

$C \equiv ap + b \pmod{26}$

$C - b \equiv ap \pmod{26}$

$\bar{a}(C-b) \equiv \bar{a}ap \pmod{26}$

$\bar{a}(C-b) \equiv p \pmod{26}$

$P \equiv \bar{a}(C-b) \pmod{26}$

RSA Encryption

Key: (n, e) 00 = A, 01 = B, etc.

M: message in sequence of ints

each int block: $C = M^e \bmod n$

ex: → Key (2537, 13) STOP → 18 19 14 15

$C = M^e \bmod 2537$

$1819^{13} \bmod 2537 = 2081$

$1415^{13} \bmod 2537 = 2182$

encrypted: 2081 2182

RSA decryption

ex: 0981 0461

$n = 43 \cdot 59$ & exponent 13 (e = 13)

K: (43·59, 13)

inverse of 13 modulo 42·58 = 2436 is d = 937

$M = C^{937} \bmod 2537$

$0981^{937} \bmod 2537 = 0704$

& $0461^{937} \bmod 2537 = 1115$

0704 1115

Chapter 5

Principle of mathematical induction

Prove that P(n) is true for all integers n, we complete these steps:

- Basis Step: Show that P(1) is true. or P(b) → starting point
- Inductive Step: Show that $P(k) \to P(k+1)$ is true for all positive integers k.

Can be expressed as: $(P(1) \wedge \forall k (P(k) \to P(k+1))) \to \forall n P(n)$

Proving summation formula

ex: $\sum_{i=1}^{n} = \frac{n \cdot (n+1)}{2}$

Basic Step: P(1) is true

sin $1 \cdot 2 / 2 = 1$

inductive step: assume true for P(k)

$\sum_{i=1}^{k} \frac{k \cdot (k+1)}{2}$

$1 + 2 + \dots + k + (k+1) = \frac{k \cdot (k+1)}{2} + (k+1)$

$= \frac{k \cdot (k+1) + 2 \cdot (k+1)}{2}$

$= \frac{(k+1)(k+2)}{2}$

Proving disibility results

ex: $3 | n^3 - n$ $\forall n \in \mathbb{Z}^+$

Basis step: P(1) is true $1^3 - 1 = 0$ $3|0$

Inductive step: assume P(k) holds.

$3 | k^3 - k$

$(k+1)^3 - (k+1)$

$= (k^2 + 2k + 1) \cdot (k+1) - (k+1)$

$= (k^3 + 2k^2 + k + k^2 + 2k + 1) - (k+1)$

$= (k^3 + 3k^2 + 3k + 1) - (k+1)$

$= (3k^2 + 3k) + (k^3 + 1) - (k+1)$

$= 3 \cdot (k^2 + k) + (k^3 - k)$

Proving inequalities

ex: $n < 2^n$ $\forall n \in \mathbb{Z}^+$

Basic step: P(1) is true since $1 < 2^1 = 2$

inductive step: Assume P(k) holds i.e $k < 2^k$

Show P(k+1) holds since $k < 2^k$

$k + 1 < 2^k + 1 \leq 2^k + 2^k = 2 \cdot 2^k = 2^{k+1}$

∴ $n < 2^n$ holds for $\forall n \in \mathbb{Z}^+$

ex: $2^n < n!$ $\forall n \in \mathbb{Z}^+$ $n \geq 4$

Basic Step: P(4) is true since

$2^4 < 4!$ i.e. $16 < 24$

Inductive step: Assume P(k) holds i.e

$2^k < k!$ ∴ True

$2^{k+1} < (k+1)!$

$2^k \cdot 2 < (k+1) \cdot k!$ (≥ 5)

Number of subsets of a finite set

Show that if S is a finite set with n elements, where n is a nonnegative integer, then S has 2^n subsets.

Basic step: P(0) is true $2^0 = 1$

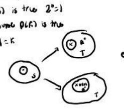
Inductive step: assume P(k) is true

2^k subsets $|S| = k$

Prove P(k+1)

$|S| = k+1$

$2^{k+1} = 2^k \cdot 2$



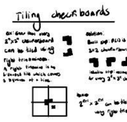
Tiling checkboards

ex: Show that every $2^n \times 2^n$ checkerboard can be tiled using right triominoes. A right triomino is an L-shaped tile which covers 3 squares at a time.

Solution: Basis step: P(1) is true 2x2 checkerboards

inductive step: assume P(k) is true for every $2^k \times 2^k$ checkerboard

$2^{k+1} \times 2^{k+1}$ can be tiled using right triominoes



Guidelines

1. $\forall n \geq b$, P(n) for a fixed int b
2. write out "Basis step", show P(b) is true
3. write "inductive step"
4. write "assume P(k) is true for an arbitrary fixed integer $k \geq b$"
5. state what needs to be proven P(k+1)
6. prove P(k+1) using assumption P(k); make sure its valid for all elements in domain
7. clearly identify conclusion
8. after that P(n) is true for all ints n with n ≥ b.

Strong induction & well-ordering

→ second principle of mathematical induction & complete induction

Strong induction: to prove that P(n) is true for all positive integers n, where P(n) is a propositional function

Steps:

Basis step: verify that P(1) is true

inductive step: Show $[P(1) \wedge P(2) \wedge \dots \wedge P(k)] \to P(k+1)$ holds for all positive integers k.

ex: Show that if n is an int greater than 1, then n can be written as the product of prime.

solution:

Basis step: P(2) is true since 2 is prime

inductive step:

inductive hypothesis: P(j) is true for all integers j with $2 \leq j \leq k$. To show that P(k+1) must be true under this assumption, two cases need to be considered

- if k+1 is prime, then P(k+1) is true
- Otherwise, k+1 is composite

ex: Prove that every amount of postage of 12 cents or more can be formed using just 4-cent & 5-cent stamps

Basis step: P(12), P(13), P(14) & P(15) hold

Inductive step: inductive hypothesis states that P(j) holds for $12 \leq j \leq k$, where $k \geq 15$. Assuming the inductive hypothesis, it can be shown that P(k+1) holds.

- P(k-3) holds since $k - 3 \geq 12$. To form postage of k+1 cents, add a 4-cent stamp to the postage for k-3 cents. hence P(n) holds for all $n \geq 12$.

→ mathematical induction can be used.

well-ordering property

Def: every nonempty set of nonnegative integers has a least element.

ex: the set of finite strings over an alphabet using lexicograph ordering is well ordered

ex: division algo $a = dq + r$

Set S: $\{a - dq\}$

Set is nonempty since $-dq$ can be made as large as needed

S has at least one element

$r = a - d \cdot q_0$ r is nonnegative

Recursive definitions & structural induction

a recursive or inductive definition of a function consists of two steps.

→ Basis step: specify value of the function at zero.

→ Recursive step: give a rule for finding its value at an integer from its values at smaller integers.

ex:

$f(0) = 3$

$f(n+1) = 2 \cdot f(n) + 3$, $f(1) = 9$, $f(2) = 21$, $f(3) = 45$, $f(4) = 93$.

fibonacci numbers: {0, 1, 1, 2, 3, 5, }

Lamé's theorem: The number of divisions used by the euclidian algorithm to find gcd(a,b) is less than or equal to five times the number of decimal digits in b.

recursive definition of sets:

basis step specifies an initial collection of elements

recursive step gives the rules for forming new elements in the set.

ex:

B.S.: $3 \in S$ {3, 6, 9, 12, ...}

R.S.: if $x \in S$ & $y \in S$, then x+y in S

Strings

The set Σ^* of strings over the alphabet Σ

B.S.: $\lambda \in \Sigma^*$ (λ is the empty string)

R.S.: If w is in Σ^* & x is in Σ, then $wx \in \Sigma^*$

concatination

B.S.: If $w \in \Sigma^*$, then $w \cdot \lambda = w$

R.S.: If $w_1 \in \Sigma^*$ & $w_2 \in \Sigma^*$ & $x \in \Sigma$, then $w_1 \cdot (w_2 x) = (w_1 \cdot w_2) \cdot x$.

length

B.S.: $l(\lambda) = 0$

R.S.: $l(wx) = l(w) + 1$ if $w \in \Sigma^*$ & $x \in \Sigma$.

well-formed formulae in propositional logic

ex: $((p \vee q) \to (q \wedge F))$ → well-formed

$pq \wedge$ → not well-formed

rooted trees + binary trees

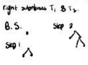
Basis step: single vertex r is a rooted tree

recursive step: $T_1, T_2, \dots, T_n$ are disjoint rooted trees with roots $r_1, r_2, \dots, r_n$ respectively, etc.

Full binary trees

Basis step: full binary tree consisting of only a single vertex r.

recursive step: If T_1 & T_2 are disjoint full binary trees, there is a full binary tree, denoted by $T_1 \cdot T_2$, consisting of a root r together with edges connecting the root to each of the roots of the left & right subtrees T_1 & T_2.



Structural induction

Def: to prove a property of the elements of a recursively defined set, we use structural induction.

B.S.: show that the result holds for all elements specified in the basis step of the recursive definition.

Recursive step: show that if the statement is true for each of the elements used to construct new elements in the recursive step of the definition, the result holds for these new elements.

Full binary trees → FBT

Def: height h(T) of a full binary tree T is defined recursively as follows:

B.S.: height of a full binary tree T consisting of only a root is h(T) = 0

R.S.: if T_1 & T_2 are FBTs, the the FBT $T = T_1 \cdot T_2$ has height $h(T) = 1 + \max(h(T_1), h(T_2))$

number of vertices

B.S.: num of vertices of FBT T consisting of only a root r is n(T) = 1

R.S.: If T_1 & T_2 are FBTs, then the FBT $T = T_1 \cdot T_2$ has the number of vertices $n(T) = 1 + n(T_1) + n(T_2)$

Theorem: if T is a FBT, then $n(T) \leq 2^{h(T)+1} - 1$.

Proof: structural induction

B.S.: FBT consists only of a root, n(T) = 1 & h(T) = 0. Hence, $n(T) = 1 \leq 2^{0+1} - 1 = 1$

R.S.: Assume $n(T_1) \leq 2^{h(T_1)+1} - 1$ & also $n(T_2) \leq 2^{h(T_2)+1} - 1$ whenever T_1 & T_2 are FBT.

Generalized induction

prove results about sets other than the integers that have the well-ordering property.

Recursive algorithms

Definition: An algorithm is called recursive if it solves a problem by reducing it to an instance of the same problem with smaller input. For the algorithm to terminate, the instance of the problem must eventually be reduced to some initial case for which the solution is known.

ex: factorials, powers, gcd, modular exponentiation, binary search

Proving recursive algorithms correct

ex:

Power (a: nonzero real number, n: nonnegative integer)

if n = 0 then return 1

else return a · power (a, n−1)

{output is a^n}

Sol:

B.S.: $a^0 = 1$ is true

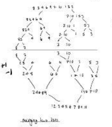
I.S.: Assume $power(a, k) = a^k$

$power(a, k+1) = a \cdot power(a, k) = a \cdot a^k = a^{k+1}$

merge sort

iterative splitting of a list, sublist is a balanced binary tree, at each step a pair of sublists is successively merged into a list with the elements in increasing order. the process ends when all the sublists have been merged.

The succession of merged lists is represented by a binary tree



merging two lists

recursive merge sort

procedure mergesort (L = $a_1, a_2, \dots, a_n$)

if n > 1 then

m := ⌊n/2⌋

$L_1 := a_1, a_2, \dots, a_m$

$L_2 := a_{m+1}, a_{m+2}, \dots, a_n$

L := merge(mergesort(L_1), mergesort(L_2))

→ can do smth similar with 2 sorted lists (two routine merge)

Complexity of merge sort

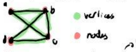
$O(n \log n)$

Chapter 9

Graph & Graph models

graph G = (V, E): consists of a set V of vertices (or nodes) & a set E of edges.

Each edge has either one or two vertices associated with it, called endpoints. An edge is said to connect its endpoints.



→ no loop, no double edge

Simple graph: each edge connects two different vertices & no two edges connect the same pair of vertices

Multigraphs: may have multiple edges connecting the same two vertices. {U, V} is an edge multiplicity m.

loop: edge that connects a vertex to itself

Pseudograph: may include loops & multiple edges connecting the same pair of vertices.

Pseudograph →

directed graphs (digraph): consists of a nonempty set V of vertices/nodes & a set E of directed edges (or arcs). Each edge is associated with an ordered pair of vertices. The directed edge associated with the ordered pair (u, v) is said to start at u & end at v.

Simple directed graph: no loops & no multiple edges.

directed multigraph: multiple directed edges.

Graph terminology

Type	Edges	multiple edges allowed?	loops allowed?
simple graph	undirected	no	no
multigraph	undirected	yes	no
Pseudograph	undirected	yes	yes
simple directed graph	directed	no	no
directed multigraph	directed	yes	yes
mixed graph	directed & undirected	yes	yes

applications

social networks
- relationships between people or groups
- vertices are individuals/organizations edges are relationships between them

communications networks
- collaboration graphs

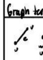
information networks
- different types of networks that link different types of information

software design
- dependencies between modules of a software application
- the other representing restrictions in the execution of statements in computer programs
- Precedence graph

transportation networks
- directed edges are used
- airline networks, road networks

Biological networks
- niche overlap graphs (competition between species)
- Protein interaction network

Graph terminology & special types of graphs



e is incident with the vertices u & v; e connects to u & v; * undirected graph u & v are neighbours

N(v) → set of all neighbours of v

deg(v) → in an undirected graph its the number of edges incident with it. * a loop is 2 degrees.

handshaking theorem

If G = (V, E) is an undirected graph with m edges, then $2m = \sum_{v \in V} \deg(v)$

theorem: an undirected graph has an even number of vertices of odd degree

directed graphs

directed edge (u, v) starts at u & ends at v

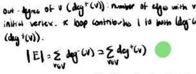
u is the initial vertex & is adjacent to v & v is the terminal (or end) vertex of this edge & is adjacent from u. the initial & terminal vertices of a loop are the same.

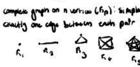
in-degree of a vertex v ($\deg^-(v)$): number of edges which terminate at v.

out-degree of v ($\deg^+(v)$): number of edges with v as their initial vertex. * loop contributes 1 to both ($\deg^-(v)$) & ($\deg^+(v)$).

$|E| = \sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v)$

complete graph on n vertices (K_n): simple graph that contains exactly one edge between each pair of distinct vertices.

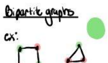




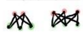
n-dimensional hypercube or n-cube, Q_n: graph with 2^n vertices representing all bit strings of length n.

Bipartite graphs

ex:

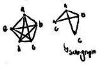


complete bipartite



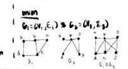
two subsets V_1 of size m & V_2 of size n s.t there is an edge from every vertex in V_1 to every vertex in V_2.

new graphs from old

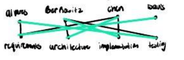


union

$G_1 = (V_1, E_1)$ & $G_2 = (V_2, E_2)$

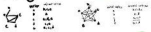


Biportite graphs & matchings

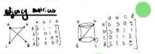


Representing graphs & graph isomorphism

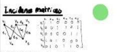
adjency list



adjency matrices



Incidence matrices

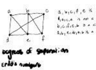


Isomorphism of graphs

The simple graphs $G_1 = (V_1, E_1)$ & $G_2 = (V_2, E_2)$ are isomorphic if there is a one-to-one & onto function f from V_1 to V_2 with the property that a & b are adjacent in G_1 if & only if f(a) & f(b) are adjacent in G_2, for all a & b in V_1. Such a function f is called an isomorphism. 2 simple graphs that aren't isomorphic are called nonisomorphic.

Connectivity

Path: sequence of edges that begins at a vertex of a graph & travels from vertex to vertex along edges of the graph. As the path travels along its edges, it visits the vertices along this path, that is, the endpoints of these.



a, b, c, f, e is a simple path of length 4; d, e, c, a is not a path since e is not connected to c; b, c, f, e, b is a circuit of length 4; a, b, e, d, a, b is a path of length 5, but not a simple

> degrees of separation

* erdős numbers

* Bacon numbers

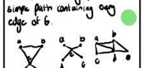
- An undirected graph is called connected if there is a path between each pair of vertices otherwise it is disconnected
- A directed graph is strongly connected if there is a path from a to b & a path from b to a whenever a & b are vertices in the graph
- A directed graph is weakly connected if there is a path between every two vertices in the underlying undirected graph, which is the undirected graph obtained by ignoring the directions of the edges of the directed graph.

* we can use the adjency matrix of a graph to find the number of paths between two vertices in the graph.

Euler & Hamiltonian graphs

Euler circuit: in a graph G is a simple circuit containing every edge of G.

Euler path: in a graph G is a simple path containing every edge of G.



has { Euler circuit: G_1; Euler path: G_2

necessary condition: Circuit terminates where it started

- degree of every other vertex must also be even

Hamilton path: simple path in a graph G that passes through every vertex exactly once.

Hamilton circuit: simple circuit in a graph G that passes through every vertex exactly once.

* no simple necessary & sufficient conditions are known for hamilton circuit

* G simple graph & $n \geq 3$ vertices s.t. $\forall v \in V$: $\deg(v) \geq n/2$, G has Hamilton circuit

* G simple graph & $n \geq 3$ vertices s.t. $\deg(u) + \deg(v) \geq n$ for every pair of nonadjacent vertices, then G has a Hamilton circuit

Dijkstra's algorithm

L(v): current best known distance from a to vertex v.

S: set of vertices whose shortest distance is already finalized.

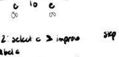
w(u, v): weight of the edge from u to v

Predecessor: previous vertex used to reach v on the current best path.

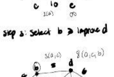
relaxation rule: for each neighbor v of the selected vertex u: If $L(u) + w(u,v) < L(v)$, then update L(v).

(L(u): already final because u has just been added to S; w(u,v): is the edge weight from u to v; * if the new route is shorter, replace the old label & store u as predecessor)

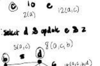
step 0: initialize labels



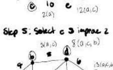
step 1: select a & relax its neighbors



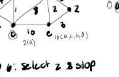
Step 2: select c & improve label e



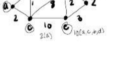
step 3: Select b & improve d



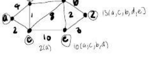
Step 4: select d & update e & z



Step 5: Select e & improve z



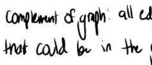
Step 6: select z & stop



adds

neighbourhoods: neighbourhood of a vertex is the vertices adjacent to it

hypercubes: an n-dimensional hypercube or n-cube, Q_n, is a graph with 2^n vertices representing all bit strings of length n, where there is an edge between two vertices that differ in exactly one bit position.



Complement of graph: all edges that could be in the graph but isnt

Powers of a matrix:

ex: $(A^2)_{11} = 1$ $A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ $A^2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$

$A \to B \to A$

$(A^k)_{ij}$ = # of walks of length k from vertex i to vertex j

Chapter 10

Introduction to trees

tree: connected undirected graph with no simple circuits.
forest: graph that has no simple circuit but is not connected
Theorem: an undirected graph is a tree if & only if there is a unique simple path between any two of its vertices.
rooted tree: a tree in which one vertex has been designated as the root & every edge is directed away from the root.

* An unrooted tree is converted into different rooted trees when different vertices are chosen as the root.

Parent, siblings, children, descendants
internal vertices: anything that's not a leaf

Subtree at x: x as the root plus descendants
m-ary tree: rooted tree where every internal vertex has no more than m children. full m-ary tree is when every internal vertex has exactly m children.

binary tree: m-ary tree with $m=2$

ordered rooted tree: rooted tree where the children of each internal vertex are ordered

binary tree
↳ left child
& right child

Theorem: a tree with n vertices has n-1 edges.

Theorem: a full m-ary tree with i internal vertices has $n = mi + 1$ vertices

Theorem:

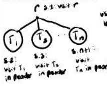
1.	2.	3.
n vertices	i internal vertices	l leaves
$i = (n-1)/m$ internal vertices	$n = mi+1$ vertices	$n = (ml-1)/(m-1)$ vertices
$l = ((m-1)\cdot n)+1$ leaves	$l = (m-1)\cdot i + 1$ leaves	$i = (l-1)/(m-1)$ internal vertices

level of a vertex v: in a rooted tree is the length of the unique path from the root to this vertex.
height of a rooted tree: maximum of the levels of the vertices.
rooted m-ary tree of height h is balanced if all leaves are at levels h or h-1.

Theorem: There are at most m^h leaves in an m-ary tree of height h.

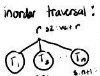
Tree traversal

Preorder traversal:


r s.1: visit r
T1
T2
Tn
s.2: visit T1 in preorder
s.3: visit T2 in preorder
s.n+1: visit Tn in preorder

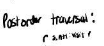
visit left to right

inorder traversal:



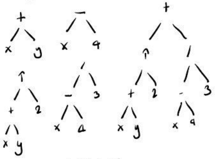
visit left to right

Postorder traversal:


r s.n+1: visit r
T1
T2
Tn
s.1: visit T1 in postorder
s.2: visit T2 in postorder
s.n: visit Tn in postorder

visit left to right

expression trees:
$((x+y)\uparrow 2) + ((x-4)/3)$



$+ - * 2 3 5 / \uparrow 2 3 4$
$+ - * 2 3 5 / 8 4$
$+ - * 2 3 5 2$
$+ - 6 5 2$
$+ 1 2$
3

pedmas ahh (operators first)

$7 2 3 * - 4 \uparrow 9 3 / +$
$7 6 - 4 \uparrow 9 3 / +$
$1 4 \uparrow 9 3 / +$
$1 9 3 / +$
$1 3 +$
4

operands first

Spanning trees

spanning tree: a spanning tree of a simple graph G is a subgraph of G that is a tree containing every vertex of G.
theorem: simple graph is connected if & only if it has a spanning tree.

Depth-first search

goal: build a spanning tree for a connected simple graph.

1. choose a vertex of the graph as the root
2. form a path by adding vertices & edges
3. if the path goes through all vertices of the graph, the tree consisting of this path is a spanning tree.
4. if not spanning go back a vertex maybe even start a new path from there & don't give up like a big fat loser!!!
5. redo until all vertices are included in the spanning tree

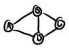
Breadth-first search

1. choose a vertex as a root
2. add all edges incident to this vertex & the other endpoint of each of these edges
   * these vertices are at level 1.
3. for each vertex added at the previous level, we add each edge incident to this vertex, as long as it does not produce a simple circuit. The new vertices we find are the vertices at the next level.
4. we continue in this manner until all the vertices have been added & we have a spanning tree.

Graphs Dijkstra MAT frameworks

algorithmic link: BFS or DFS can find all vertices in one component. (BFS ↳ Breadth-first search; DFS ↳ Depth-first search)
types of graphs: directed, undirected, weighted
complete graph: every pair of vertices has an edge
DAG: Directed acyclic graph. useful for prerequisites, scheduling & dependency ordering

adjacency list:
A: B, D
B: A, C, D
C: B, D
D: A, B, C



matrix

$$\begin{array}{c|cccc} & a & b & c & d \\ \hline a & 0 & 1 & 0 & 1 \\ b & 1 & 0 & 1 & 1 \\ c & 0 & 1 & 0 & 1 \\ d & 1 & 1 & 1 & 0 \end{array}$$

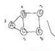
weighted matrix: 0 or ∞ when no edge exists vs weight when it exists

adjacency list (weighted):
A: (B,2), (D,1)
B: (A,2), (C,2), (D,5)
C: (B,2), (D,4)
D: (A,1), (B,5), (C,4)

BFS

1. BFS explores in layers from a starting vertex
2. It uses a queue to process vertices in discovery order
3. In an unweighted graph BFS gives shortest path in number of edges

Intuition: explore neighbors, then neighbors of neighbors

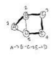


applications:
shortest path (unweighted)
connected components
level-order exploration
complexity

DFS

1. DFS follows one path deeply before backtracking
2. It can be implemented recursively or with an explicit stack
3. DFS is especially useful for structure: cycles, components & topological ordering

Intuition: explore, deepen, backtrack



applications:
cycle detection
connected components
topological ordering

maze/search problems
complexity
* DFS does not guarantee shortest paths in unweighted graphs. BFS does

BFS algorithm
- uses queue: 1st discovered, 1st processed
- mark a vertex when it is discovered
- parent relation reconstructs a BFS tree & shortest unweighted paths.

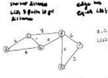
DFS algorithm
- recursive DFS uses the call stack. iterative DFS uses an explicit stack.
- If the graph is disconnected, DFS produces a forest rather than one tree
- DFS supports deeper analysis such as detecting back edges & cycles.

Dijkstra's algorithm

shortest path problem:
input
[illegible]
output
shortest distance L(z) & path to get distance

BFS is not enough because it assumes each edge has equal cost

* greedy choice
positive weights
predecessors



a, c, b, d, e, z
L(z) = 13
↳ shortest path a to z.

backward chain:
z←e←d←b←c←a
reverse it:
a→c→b→d→e→z

visual summary & references

Graph model V, E → representation list/matrix → BFS Queue/layers → DFS Stack/recursion → Dijkstra labels/weights

SSSP: single-source shortest paths
PQ: priority queue
BFS/DFS tree: traversal tree built by parent links

ADJ: adjacency list or adjacency relation
∞: Infinity: no known path yet.

Program correctness, preconditions, postconditions, & invariants

* understand first, visualize second, practice third, master forever

foundation

- a program is correct only relative to a precise specification.
- discrete mathematics provides the logic used to state & prove properties
- correctness requires more than "the program worked on my test cases".

Testing: run program on selected inputs

Static analysis: examine code without executing every path. find defects & sus patterns
formal proof: use logic to show that every condition satisfying the precondition satisfies the postcondition & remains safe

correctness: relation between code & specification
I: input domain
O: required result
R: allowed behaviour

Hoare triple
P – Precondition ↳ true before execution
C – command ↳ true after execution
Q – Postcondition

P – partial correctness
↳ doesn't prove program termination
T – Total correctness
↳ requires variant used to establish progress

Preconditions & postconditions

Precondition:
property that must be true before calling the function
caller: must satisfy the precondition before invoking the subprogram

Complete correctness
partial correctness
↓
termination

Postcondition:
property guaranteed after the function returns

Partial correctness
If the program terminates, then the postcondition is true.
It does not guarantee termination

Invariants & loops

loop invariants
Def: statement that remains true before, during & after every iteration.

Proof obligations → exit
invariant + loop stops ⇒ postcondition
↓ Initialization
- invariant true before first iteration
↘ Preservation
- If true before one iteration, still true afterward

Induction connection
logic proofs = mathematical induction
initialization = base case
Preservation = inductive step
invariant = induction hypothesis

loop variant
purpose: prove termination
requirements:
- non-negative
- strictly decreases
- well-founded (natural numbers)
if it decreases every iteration, loop must terminate.

Designing an invariant
1. start from the postcondition
2. replace the full range with the processed region.
3. add bounds/safety facts.
4. check initialization, preservation, & exit
5. add a decreasing variant.

common invariant pattern
for accumulator loops:
Describe
- processed region
- accumulator meaning
- why the processed region becomes the whole input at termination.

common invariant mistakes
1. too weak
2. too strong
3. off-by-one
4. missing frame condition
5. missing bounds
6. missing progress (variant)

Frame conditions
state what does not change during the loop.
especially useful for arrays.
ex: everything outside [illegible] remains unchanged

linear search invariant
Invariant: Everything before I has already been checked & does not contain the key.

Binary search
Must know:
Precondition → array sorted
Invariant → If key exists, it is still inside: low..high
variant → high-low+1 decreases every iteration

euclid algorithm
invariant
GCD never changes
variant
b decreases
exit
GCD(A,0) = A
∴ answer is A

verification conditions
GNAT prove proves:
- overflow safety
- index validity
- division safety
- contracts
- invariants
- initialization

flow analysis vs proof
Flow analysis:
looks for:
- uninitialized reads
- dependencies
- ineffective assignments

Proof:
shows contracts assertions invariants are mathematically true

SPARK vs Ada
Ada
general programming language
SPARK
- restricted subset
- proof annotations
- GNAT prove

why types matter
use precise types to prevent illegal states, reduce proof obligations, improve correctness

five rules to memorize
1. correctness is relative to a specification
2. callers establish preconditions
3. implementation establishes postconditions
4. loop invariants prove preservation; loop variants prove progress/termination
5. Precise types & contracts simplify proofs & improve safety.

algorithm choice
all edges same choice? → BFS
weighted graph? ↗ all weights ≥ 0 → Dijkstra
↘ any negative edge → use Bellman-Ford
↳ slower tho

Bellman-ford
everything is fine except a negative cycle length
like Dijkstra except you find the smallest non-negative path

Program

```
BellmanFord(G,S)
 for each vertex v:
   d[v] = ∞
   pred[v] = null
 d[S] = 0
 repeat |V|-1 times:
   for each edge (u,v) in E:
     relax(u,v)
 for each edge (u,v) in E:
   if d[u] + w(u,v) < d[v]:
     report reachable negative weight cycle
 return d, pred
```

bridge: edge whose removal disconnects the graph

max amount of edges in simple graph:
n vertices
↳ $\#E = \dfrac{n\cdot(n-1)}{2}$

topological sorting:
is a linear ordering if U→V, then U appears before V